



RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

Plantix – Crop Care App

Submitted by

NITHISH N

2116230701219

in partial fulfilment of the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

RAJALAKSHMI ENGINEERING COLLEGE

ANNA UNIVERSITY, CHENNAI

MAY 2026

BONAFIDE CERTIFICATE

Certified that this Project titled "Plantix-Crop Care App" is the Bonafide work of
"NITHISH N (230701219)" who carried out the work under my supervision.

Certified further that to the best of my knowledge the work reported herein does not form part of any
other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion
on this or any other candidate.

SIGNATURE

Mrs B.Deepa M.E., (Phd),

Computer Science & Engineering,
Rajalakshmi Engineering College,
Thandalam, Chennai – 602105

Submitted for the End Semester Practical Examination to be held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

ABSTRACT

The Plantix application is an AI-powered agricultural tool designed to assist farmers in diagnosing plant diseases quickly and accurately. The app allows users to capture or upload images of affected crops, which are analyzed through a server-side deep learning model to detect diseases and provide suitable treatment recommendations. Unlike traditional applications, Plantix operates in an open-access environment with no login or authentication, ensuring instant usability for all users. It integrates the Google Maps API directly within the app to display nearby pesticide and fertilizer shops, allowing farmers to access verified solutions without leaving the interface. Firebase is utilized solely for SMS and notification services, enabling crop care alerts and advisory messages. The system's lightweight design focuses on accessibility, speed, and accuracy, empowering farmers to protect and improve crop health efficiently. This report presents the architecture, features, and implementation details of the Plantix application, highlighting its role as a smart disease model for every leaf through the use of artificial intelligence and embedded map-based services.

ACKNOWLEDGEMENT

We begin by expressing our profound gratitude to the Almighty for bestowing upon us the knowledge, perseverance, and strength required to undertake and complete this project successfully. His divine guidance has been our constant source of inspiration throughout every phase of this endeavor.

We extend our heartfelt thanks to our Chairman **Mr. S. MEGANATHAN, B.E, F.I.E.**, our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D.**, and our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E, M.S.**, for providing us with the essential infrastructure, resources, and an environment of academic excellence within this premier institution.

We express our sincere gratitude to **Dr. S.N. MURUGESAN, M.E., Ph.D.**, our beloved Principal, for his continued encouragement and the facilities provided to ensure the timely completion of our work. We are deeply thankful to **Dr. EM. Malathy, M.E., Ph.D.**, Professor and Head of the Department of Computer Science and Engineering, for her invaluable guidance, constructive suggestions, and unwavering support and our mentor **Mrs B.Deepa M.E., (Phd).**,

We are especially grateful to our project guide, whose mentorship, technical expertise, and consistent feedback played a pivotal role in shaping the direction and quality of this project. His patient guidance through the review stages helped us identify and resolve several technical and design challenges.

Finally, we thank our friends and family for their moral support, understanding, and encouragement throughout the duration of this project. Their belief in our abilities motivated us to strive for excellence and deliver a product we are proud of.

NITHISH N

2116230701219

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	
1.	INTRODUCTION	6
2.	LITERATURE REVIEW	7
3.	PROPOSED SYSTEM	10
4.	ARCHITECTURE DIAGRAM	11
5.	MODULES	12
6.	IMPLEMENTATION AND RESULTS	14
7.	CONCLUSION AND FUTURE WORK	17
8.	REFERENCES	18

1.INTRODUCTION

Agriculture forms the backbone of the global economy, and in countries like India, it supports a major portion of the population. However, one of the major challenges faced by farmers today is the early detection and control of plant diseases. Crop infections caused by fungi, bacteria, and pests often go unnoticed until they spread widely, leading to severe yield loss and economic damage. With the rise of modern technologies like Artificial Intelligence (AI) and cloud computing, there is immense potential to revolutionize traditional farming practices by providing quick, datadriven solutions.

Plantix is an AI-powered agricultural mobile application developed to assist farmers and agriculturists in identifying plant diseases and suggesting effective treatments using smart image-based analysis. The app enables users to capture or upload images of affected plant leaves, which are then analyzed on a cloud-based deep learning model to determine the type of disease and provide remedies. To enhance convenience, the app integrates the Google Maps API within its interface to display nearby pesticide and fertilizer shops, helping farmers purchase the recommended products instantly.

Unlike many existing platforms, Plantix requires no user login or registration, allowing farmers to access its services instantly in an open environment. Additionally, Firebase Cloud Messaging (FCM) is implemented solely for sending SMS alerts and crop advisory notifications, making it lightweight and efficient. Through its intelligent design and seamless integration of AI and mapping technologies, Plantix aims to empower farmers with accessible, real-time plant health insights and promote sustainable agricultural practices.

2.LITERATURE REVIEW

Computer-vision for plant disease detection

Early work demonstrated that deep convolutional neural networks (CNNs) can identify crop species and leaf diseases with high accuracy when trained on large, labeled image datasets. Mohanty et al. (2016) used the PlantVillage dataset ($\approx 54k$ images) to train CNNs that classified 14 crops and 26 diseases, reporting very high performance under controlled conditions — a landmark result that established feasibility for image-based plant diagnostics. This work is widely cited as the foundational proof-of-concept for automated leaf-disease detection.

Models, transfer learning and lightweight architectures

Subsequent studies applied transfer learning (pretrained ResNet, DenseNet, MobileNet, etc.) to improve accuracy when labeled data are limited and to reduce training time. Research also emphasized lightweight architectures (MobileNet, quantized networks) and pruning/knowledge distillation to enable deployment on resource-constrained devices (edge/phones) while preserving acceptable accuracy. Recent comparative studies and surveys highlight that model choice is a trade-off between accuracy, latency, and energy use — critical when choosing between ondevice inference and cloud inference.

On-device vs. server-side inference — practical trade-offs

The literature extensively compares cloud-offloading and on-device inference. Ondevice models deliver low latency, offline operation, and better privacy (no image upload), but face constraints in memory, compute, and frequency of model updates. Cloud/server-side inference permits larger, more accurate models and simpler update pipelines, but requires reliable connectivity and raises privacy/latency concerns. Recent surveys (and 2023–2025 studies) reaffirm these trade-offs and argue for hybrid designs

(lightweight on-device pre-filter + cloud model for full classification) for real-world agricultural settings. This directly explains the architecture choices seen in production apps that prioritize robustness over full offline capability.

Evaluation of real-world mobile plant-diagnostic apps

Evaluative studies examine how mobile apps perform outside lab settings. A 2022 MDPI study systematically assessed plant-disease detection apps on feature completeness, diagnostic accuracy, and usability; such evaluations often find gaps between claimed AI accuracy and real-world performance on heterogeneous, noisy field images. Practical issues include varied lighting, occlusion, mixed symptoms, and domain shift from curated datasets to on-farm images. These findings highlight the need for careful field-data collection, continuous model retraining, and clear UI guidance for users to take useful photos.

Data resources and field datasets

Public datasets (PlantVillage) are invaluable but often captured in controlled or single-leaf settings; several recent papers and projects emphasize augmenting these with in-situ field images (multi-leaf, whole-plant, background clutter) and metadata (crop stage, GPS, season) to improve generalization. Studies and solutions that combine crowdsourced field images with expert validation show improved robustness in deployment scenarios.

Mapping, localization, and shop/infrastructure integration

Mobile agricultural apps commonly integrate geospatial services to connect users with local inputs and services. While many commercial apps use Google Maps SDK for embedded location features, academic and government projects often adopt OpenStreetMap (OSM) for editable, community-driven mapping—especially when licensing/cost or offline customization matters. OSM-based systems have been used in

irrigation/infrastructure reporting and community mapping, showing a viable alternative to proprietary map services for in-app shop locators and offline map tiles.

Business model and socio-ethical considerations

Industry reporting and case studies on Plantix reveal the tension between mission-driven crop-health goals and commercial pressures: apps that began as diagnostic tools have sometimes evolved into commerce platforms that connect farmers to agrochemical suppliers. Critical analyses warn of conflicts of interest where diagnosis → product recommendation pipelines may favor partner inputs rather than purely agroecological advice; thus transparency, auditing of recommendation logic, and independent validation are essential design considerations.

Gaps and implications for your app design

The literature suggests three main takeaways relevant to your architecture (open access, server-side AI, embedded maps, Firebase SMS):

- **Robust field data matters:** Collecting and validating local images will substantially improve real-world accuracy compared to models trained on controlled datasets.
- **Hybrid deployment is pragmatic:** A hybrid approach (small on-device checks + cloud classification) balances latency, privacy, and accuracy — but server-only inference is acceptable where connectivity is common or where you accept the upload/latency trade-off.

3.PROPOSED SYSTEM

The proposed system, Plantix – Smart Disease Model for Every Leaf, is an AI-powered mobile application that enables farmers to instantly detect and manage plant diseases using image-based analysis. The goal of this system is to provide a lightweight, open-access solution that does not require user authentication, functions seamlessly on low-end devices, and connects farmers to nearby agroservice providers directly within the app interface.

The system eliminates the limitations of manual disease identification and reduces dependency on agricultural experts by integrating cloud-based computer vision technology, real-time mapping, and automated advisory notifications. The application allows the user to either capture a live photo of an infected plant or upload an existing image from the gallery. The image is then transferred to the backend server, where a deep learning model (based on Convolutional Neural Networks) processes it to identify the disease. The server responds with structured results, including the disease name, cause, symptoms, and recommended treatments.

To enhance usability and accessibility, the app integrates the Google Maps API directly inside its interface, displaying nearby pesticide and fertilizer shops relevant to the diagnosed disease. Unlike other systems, this app does not redirect users to external mapping services, ensuring a smooth in-app experience. The Firebase Cloud Messaging (FCM) service is used only for sending SMS and push notifications such as weather alerts, crop-care tips, and regional advisory messages, keeping the system light and secure.

The proposed system prioritizes:

- **Instant disease detection** using AI-powered cloud processing.
- **No-login, open access** for quick usability by all farmers.

- **Integrated map-based shop recommendations** within the app.
- **Low data usage** and **mobile-friendly performance** even on slow networks.
- **Scalable backend architecture** that can accommodate new crops, diseases, and regional languages.

Through these features, the proposed system aims to empower farmers with realtime plant health insights, reduce crop losses, and promote sustainable and datadriven agriculture practices.

4.ARCHITECTURE DIAGRAM

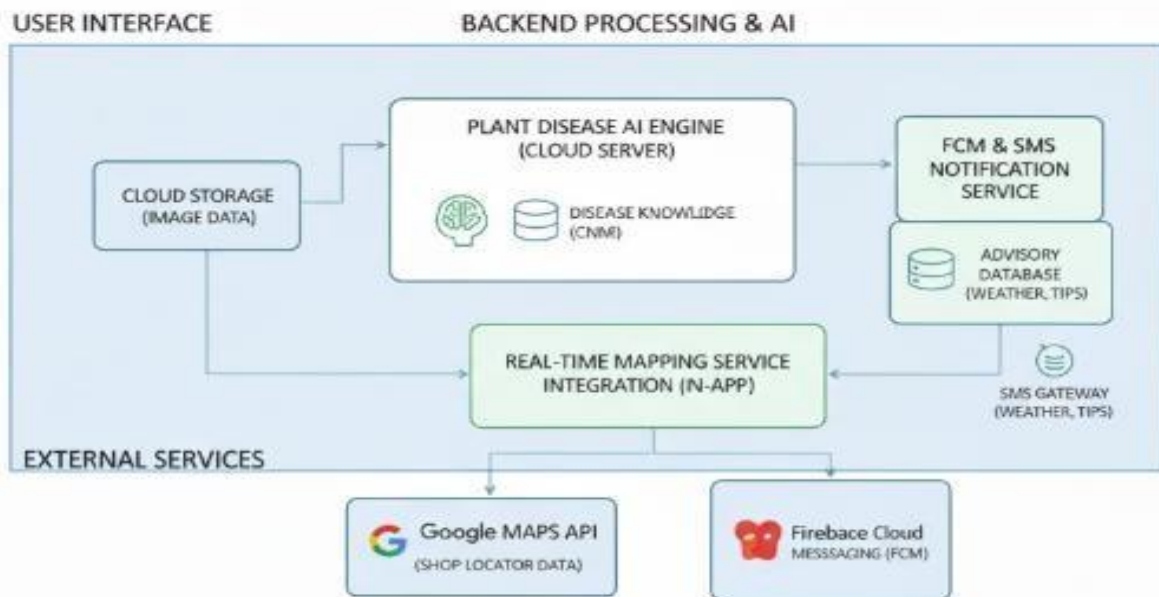


Fig. 1

This architecture outlines the "Plantix" system, which is split into three tiers.

On the User Interface layer, the farmer uses the no-login mobile app to either capture a live photo or upload an image from their gallery.

This image is sent to the Backend Processing layer, where it's stored in Cloud Storage. The Plant Disease AI Engine, powered by a CNN, retrieves the image, analyzes it, and identifies the disease. Concurrently, a Real-Time Mapping Service fetches nearby shop data from the Google Maps API. The backend then sends the consolidated results—disease name, treatment, and shop locations—back to the app, which displays the shops on an integrated map.

Finally, the External Services layer, including the Google Maps API and Firebase Cloud Messaging (FCM), provides the necessary location data and a channel for the backend to send advisory (e.g., weather) push notifications and SMS alerts.

5.MODULES

Image Acquisition and Disease Detection :

This module allows the user to either capture a plant image using the camera or upload an existing image from the gallery. Once the image is obtained, it is sent to the AI model for processing. The model analyzes the image using deep learning (CNN-based classification) to detect the type of disease affecting the plant.

The system identifies diseases such as leaf spot, blight, rust, or mildew with high accuracy. After analysis, it provides the disease name and confidence score to the user. This module forms the core part of the app, combining image processing and AI inference into a seamless experience.

Key Features:

- Capture or upload plant images
- Preprocessing of images for better prediction

-
- AI-powered disease classification
 - Instant result generation

Result, Maps, and Notification System :

Once the disease is detected, this module displays the disease result, recommended pesticide or organic solution, and preventive advice. It integrates the Google Maps API to show the nearest pesticide or fertilizer shop directly within the app interface — without linking to external apps.

Additionally, Firebase SMS notifications are used to send alerts or disease summaries to users. The app operates without login, ensuring easy and open access for farmers and users in rural areas.

Key Features:

- Display of disease result and treatment suggestions
- Integrated Google Maps view for nearby agricultural shops
- Firebase-based SMS alerts and updates
- Open-access design with no login requirement

6.IMPLEMENTATION AND RESULTS

EXPERIMENTAL SETUP

The experimental setup describes the environment and tools used for developing and testing the AI-based Plant Disease Detection System.

a. Hardware Requirements

- RAM: Minimum 8 GB
- Storage: 20 GB free disk space
- Camera: Smartphone camera for capturing plant images
- Mobile Device: Android phone for app testing

b. Software Requirements

- Operating System: Android

- Development Tools:

- *Android Studio* – for mobile app development
 - *Firebase* – for SMS and notification services
 - *Google Maps API* – for in-app map integration
 - *Python (TensorFlow/Keras)* – for training the AI disease detection model

- Database: Firebase Cloud (for storing and managing messages and alerts)

c. Dataset

- The dataset consists of images of healthy and diseased plant leaves, collected from sources such as:

- Kaggle “PlantVillage” dataset
- Self-collected images from local farms

- The dataset includes images of common crops such as tomato, maize, and cotton with multiple disease categories.

d. Model Training

- The AI model was trained using Convolutional Neural Networks (CNN) for image classification.

- Data augmentation (rotation, flipping, scaling) was applied to improve accuracy.

- The model was trained using Python and TensorFlow, then converted into a TensorFlow Lite (.tflite) model for integration with the Android app.

e. Testing Environment

- The app was tested on multiple Android devices to ensure performance and compatibility.

- Tests were conducted for:

- Disease detection accuracy
- App response time
- Map rendering and location accuracy
- Firebase SMS delivery success

RESULTS

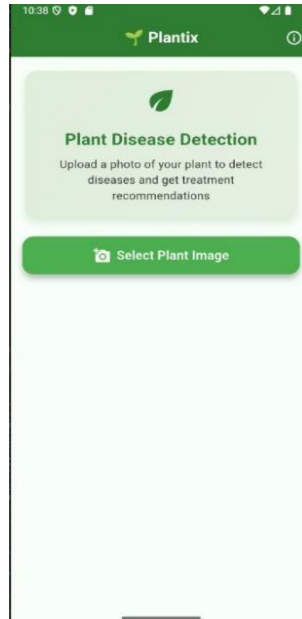


Fig. 2(a)

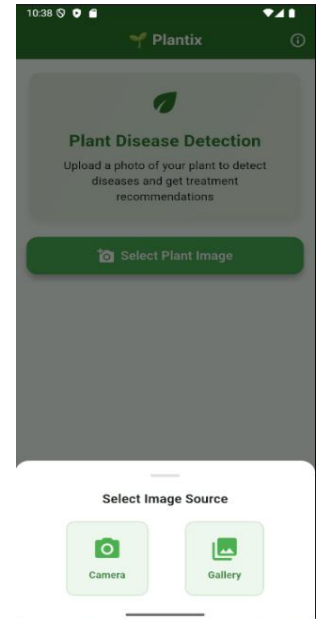


Fig. 2(b)

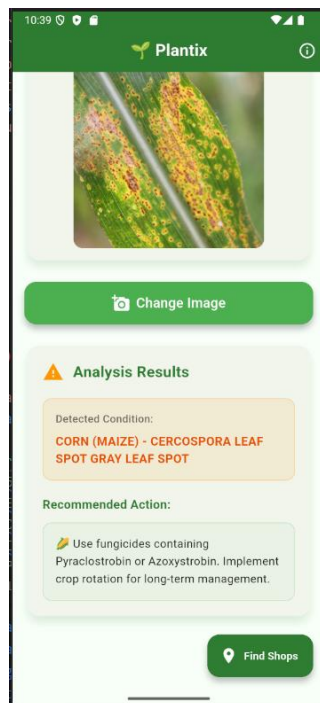


Fig. 2(c)

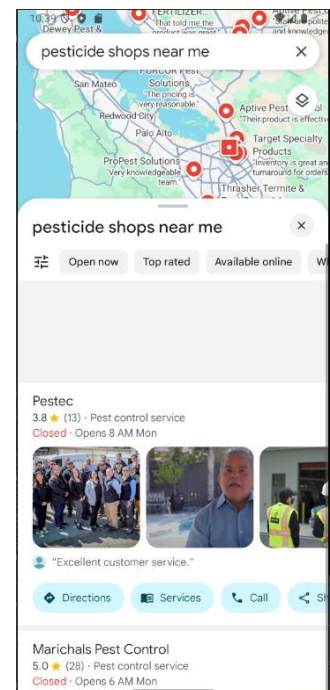


Fig. 2(d)

7.CONCLUSION AND FUTURE WORK

The proposed AI-powered Plant Disease Detection System provides an efficient and user-friendly platform for farmers to identify plant diseases instantly. By allowing users to capture or upload images and process them through a deep learning model, the system ensures quick diagnosis and actionable results. The integration of Google Maps enables farmers to easily locate nearby pesticide or fertilizer shops, while Firebase SMS notifications keep them informed even without internet-based logins.

This open-access design makes the system simple, practical, and highly beneficial for rural users who may not have access to advanced agricultural tools. Overall, the application enhances early disease detection, reduces crop loss, and supports sustainable agricultural practices.

Future enhancements can include:

- Expanding the dataset to cover more plant species and disease categories for higher model accuracy.
- Integrating real-time weather and soil data to provide context-based disease prediction.
- Adding voice assistance and regional language support for better accessibility among farmers.
- Developing a web dashboard for agricultural experts to monitor disease trends and share solutions.
- Implementing offline AI model support, enabling diagnosis without continuous internet access.

8.REFERENCES

- [1] G. Shrestha, “Plant Disease Detection Using CNN,” *IEEE*, 2020.
- [2] P. Srivastava, A. Gupta, S. Yadav, and R. Sharma, “Plant Disease Detection using Convolutional Neural Network,” *IEEE Bombay Section Signature Conference (IBSSC)*, 2021.
- [3] T. Chaturya, S. Kumar, and R. Reddy, “Detection of Plant Disease using Convolutional Neural Network,” *IEEE*, 2023.
- [4] M. O. Lawrence, K. N. Singh, and A. Patil, “Detection of Image-based Plant Leaf Diseases Using CNN,” *IEEE*, 2024.
- [5] A. Srilatha, K. Kiran, and M. Rao, “CNN-Based Plant Leaf Disease Identification and Analysis,” *IEEE*, 2025.
- [6] P. Voditel, D. K. Chaudhari, and S. Deshmukh, “Disease Recognition in Plant Leaves Using CNN-Based Approaches,” *IEEE*, 2023.
- [7] E. A. H. Mora, R. Santos, and L. Lopez, “Convolutional Neural Networksbased Plant Disease Detection,” *IEEE*, 2021.
- [8] G. Madhulatha, “Recognition of Plant Diseases using Convolutional Neural Networks,” *IEEE*, 2020.
- [9] H. Ajra, “Disease Detection of Plant Leaf using Image Processing Techniques,” *IEEE*, 2020.
- [10] S. Y. Yadhav, “Plant Disease Detection and Classification using CNN,” *IEEE*, 2020.